



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



North Korean Hackers Are Turning Developer Tools Into Malware Delivery Channels

Malware Analysis Report

Classification	TLP:CLEAR
Published	2026-06-17
Version	1.0
Author	Lost Boys Cyber
Report Type	Malware Analysis Report

TLP:CLEAR | Lost Boys Cyber | North Korean Hackers Are Turning Developer Tools Into Malware Delivery Channels - Malware Analysis Report

Published: 2026-06-17 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Sample Metadata
3. Source Review & Confidence
4. Initial Triage and Delivery Assessment
5. Static Analysis
6. Code Analysis
7. Dynamic Analysis
8. Process Tree and Execution Chain
9. Persistence and System Modification
10. Network and Infrastructure Analysis
11. Indicators of Compromise
12. MITRE ATT&CK Mapping
13. Detection Engineering
14. Threat Hunting
15. Triage and Incident Response
16. Validation and Emulation
17. Recommendations
18. References and Refresh Notes

North Korean Hackers Are Turning Developer Tools Into Malware Delivery Channels

1. Executive Summary

Lost Boys Cyber assesses with medium-high confidence that North Korea-aligned operators are expanding developer-targeted malware delivery from fake interview chats and malicious packages into a broader developer-tool abuse model. The most detailed cluster in the reviewed reporting is Proofpoint's UNK_DeadDrop: a phishing campaign observed in April-May 2026 that sent more than 250 emails to individuals at nearly 100 organizations and used actor-controlled GitHub/GitLab repositories to deliver cross-platform malware through Visual Studio Code and Cursor workflows.

The core UNK_DeadDrop infection chain weaponizes normal developer behavior. Targets receive recruitment, code-review, cryptocurrency, Foundry, exploit-archive, or AI-payment project lures, clone a repository, and open it in VS Code or Cursor. A hidden `.vscode/tasks.json`` uses ``runOptions.runOn: "folderOpen"``` to execute a platform-specific loader. The loader installs a malicious VS Code extension masquerading as a Google service and then stages either Overlord-derived Go RAT functionality on Linux/macOS or a Windows Node.js/Python credential-theft pipeline running inside the editor's Electron runtime.

The business risk is not limited to one malware family. The campaign targets developer endpoints because they are rich with browser credentials, SSH material, cloud/API tokens, wallet files, source repositories, package-publishing access, and CI/CD trust. The same intake article also links this trend to malicious VS Code marketplace extensions reported by Yeeth Security, where VSIX packages used JavaScript, Python, .NET/Roslyn execution, Microsoft Graph API, and SharePoint list queues for C2, victim registration, command delivery, and exfiltration. Even where direct IOC overlap is absent, the common strategic pattern is clear: trusted developer tools are becoming malware delivery channels.

Immediate actions are to block and retrohunt the reported C2 and sender infrastructure, inventory VS Code/Cursor/VSCodium extensions and workspace trust settings, inspect developer workstations for hidden `.vscode/tasks.json`` auto-run tasks and suspicious VSIX packages, revoke exposed wallet/API/cloud credentials, and harden repository-opening procedures for unsolicited code-review and recruitment projects. Treat a confirmed hit as a broader compromise lead, not only a malicious-extension cleanup event.

Finding	Analyst Assessment	Defensive Implication
UNK_DeadDrop used phishing plus malicious repositories	High confidence from Proofpoint reporting	Developer recruitment and code-review emails require the same scrutiny as attachment-based malware
VS Code/Cursor task auto-execution starts the chain	High confidence	Workspace trust, task approval, and Cursor behavior should be explicitly controlled
Linux/macOS payloads use Overlord-derived Go RAT binaries	High confidence from source reporting	Expect persistent WebSocket C2, wallet theft, fake password prompts, and keychain/keyring access attempts
Windows chain runs in editor/Electron context	High confidence	Code.exe/Cursor.exe may be the suspicious runtime; do not expect an obvious standalone malware process
Malicious VSIX marketplace extensions show a parallel path	Medium-high confidence	Extension governance must cover marketplace installs, not just cloned repositories
Attribution overlaps DPRK tradecraft but has caveats	Medium confidence	Focus response on TTPs and asset exposure; avoid over-weighting actor labels during triage

2. Sample Metadata

No live malware sample was provided to Lost Boys Cyber for local detonation or reverse engineering. Metadata below reflects source-reported artifacts from Proofpoint and Yeeth Security, plus defensive assessment derived from those reports.

Field	Value
Family / Cluster	UNK_DeadDrop; Overlord-derived developer-targeting tooling; related malicious VSIX backdoor activity
Malware Type	Cross-platform developer-workstation RAT, credential stealer, wallet stealer, malicious IDE extension, supply-chain intrusion enabler
Platform	Windows, macOS, Linux; VS Code, Cursor, VSCodium, VS Code Marketplace
Primary Sample SHA256	No single primary sample supplied; representative hashes listed in IOC section
Additional Hashes	Proofpoint reported hashes for tasks.json, run-update scripts, VSIX, encrypted Windows payloads, Overlord binaries, and darwin-password-prompt
File Type / Format	`.vscode/tasks.json`, shell script, VBScript, CMD, JavaScript, encrypted payload blobs, VSIX extension, Go/Mach-O/ELF binaries, .NET/PE and Python components in adjacent VSIX reporting
Delivery Theme	Recruitment, code review, crypto trading, crypto prediction, exploit archive, ERC-4626 Foundry testing, AI payments, Jupyter Notebook productivity extension
First Seen / Reported	Proofpoint activity observed April-May 2026; Proofpoint blog 2026-06-08; Yeeth Security VSIX report 2026-06-09; The Hacker News intake 2026-06-15
Analyst Confidence	Medium-high overall; high for Proofpoint-reported UNK_DeadDrop chain and IOCs, medium for relationship among all DPRK-linked developer-tool clusters
Primary Objective	Cryptocurrency wallet theft, browser credential theft, remote command execution, repository/workstation foothold, possible downstream software-supply-chain compromise
Source Basis	Proofpoint primary research, Yeeth Security primary research, The Hacker News synthesis, Lost Boys Cyber analytic enrichment

Representative UNK_DeadDrop repository lures:

Repository / Project	Theme	Reported Purpose
`pulsynk`	Crypto prediction	AI-powered cryptocurrency price prediction platform lure
`trixauvex`	Crypto trading	Trading engine and analytics platform lure
`rekt-db`	Exploit archive	Cross-chain blockchain exploit proof-of-

		concept archive lure
`forge-4626-invariants`	Foundry testing	ERC-4626 vault invariant test lure
`x402-kit`	AI payments	HTTP 402 micropayment infrastructure for AI agents lure

3. Source Review & Confidence

Source	Contribution	Confidence	Notes
Proofpoint, "Don't Fear the Repo: UNK_DeadDrop Phishing Campaign Targets Developers to Steal Cryptocurrency"	Primary infection-chain, malware behavior, targets, infrastructure, hashes, repository lures, attribution caveats	High	Most important source for UNK_DeadDrop technical detail
Yeeth Security, "Inside the DPRK-Linked Backdoor Loitering in the VS Code Marketplace"	Primary detail on malicious VS Code Marketplace extensions, Graph API/SharePoint C2, Roslyn/.NET execution, AES encryption, extension names	Medium-high	Treated as adjacent developer-tool backdoor activity; no direct public IOC overlap with UNK_DeadDrop
The Hacker News intake article	Synthesis tying UNK_DeadDrop, malicious VSIX extensions, Contagious Interview, TaskJacker, npm/Packagist activity, BlueNoroff, and broader DPRK developer targeting	Medium	Useful for landscape framing; individual claims traced to named primary sources where possible
Lost Boys Cyber analysis	Detection, hunting, triage, recommendations, ATT&CK mapping, confidence separation	Medium	Source-backed; no original sample execution in this task

Direct evidence includes Proofpoint-observed phishing volume, target sectors and geographies, malicious repository themes, IDE auto-run task usage, platform-specific loader behavior, C2 IP `23.137.105[.]75:5173`, multiple hashes, and detailed Windows/Linux/macOS credential-theft flows. Yeeth Security directly reports extension names, a shared SharePoint site GUID, a shared C2 AES key, broker domains, platform agents, and command/exfiltration behaviors.

High-confidence assessment: the campaign deliberately uses legitimate developer tooling as the execution boundary. A developer's decision to open a repository or install an extension becomes the trust event, and malicious behavior then executes inside expected IDE processes.

Lower-confidence inference: whether UNK_DeadDrop is a direct evolution of Contagious Interview, a separate North Korea-aligned team, or a subgroup reusing DPRK tradecraft remains unresolved in public reporting. This report therefore prioritizes defensive behaviors and observables over definitive actor naming.

4. Initial Triage and Delivery Assessment

The delivery model is tailored to software developers and security researchers. Lures are not generic invoices or documents; they are plausible technical assignments, crypto projects, ERC-4626 testing repositories, exploit archives, and AI-agent payment toolkits. The repositories contain credible structure and working scripts, which reduces suspicion and makes basic visual review insufficient.

Variant / Artefact	Delivery Role	Notes
Phishing email	Initial access	Uses job recruitment or peer code-review

		themes; attacker-owned sender domains masquerade as companies
GitHub/GitLab repository	Trust anchor and payload container	Hosts professional-looking project files plus hidden <code>`.vscode`</code> automation
<code>`.vscode/tasks.json`</code>	Auto-execution trigger	Defines <code>`.runOptions.runOn: "folderOpen"`</code> and launches shell/VBScript/CMD payloads
<code>`.vendor/run-update.sh`</code>	Linux/macOS loader	Installs VSIX, selects Overlord binary, removes quarantine on macOS, detaches execution, schedules cleanup
<code>`.vendor/run-update-hidden-launch.vbs`</code> and <code>`.run-update.cmd`</code>	Windows launcher	Hidden VBScript launches CMD, decodes payloads, installs VSIX, and stages encrypted JS/Python components
Malicious VSIX extension	Persistence and activation	Masquerades as a Google service in UNK_DeadDrop; adjacent activity uses Jupyter productivity extension lures
Cursor editor behavior	Execution accelerator	Proofpoint reports Cursor does not show the same trust prompt and can execute immediately on folder open

The triage priority for an enterprise is exposure scoping: identify users who received similar emails, cloned listed repositories, opened unsolicited repositories in VS Code/Cursor, installed suspicious VSIX packages, or made network connections to the reported C2 endpoints. Confirmed execution should trigger credential containment for browser-stored secrets, cryptocurrency wallets, SSH keys, cloud tokens, package registry tokens, source-control access, and CI/CD secrets.

5.1 File and Build Characteristics

No local sample archive was available. Static findings below are source-reported and should be verified against original artifacts if acquired.

Static Signal	Evidence	Analyst Value
Hidden VS Code task	<code>`.vscode/tasks.json`</code> hash <code>`c935808147f0236c81483d7bbda4b9d602f3595d5d4057f8115d39e222d1c4b`</code>	High-signal file for repository pre-open scanning
<code>`.runOn: folderOpen`</code>	Proofpoint reports task auto-runs when workspace opens	Detects abuse of an IDE feature rather than a software exploit
Cross-platform launchers	Bash for Linux/macOS, VBScript/CMD for Windows	Enables one repository to compromise heterogeneous developer fleets
VSIX package	<code>`.google-update-support.vsix`</code> hash <code>`52886aab179f26421678ff23af1b0fabf0a17ffbb534369cddbacc8008cbcd8e7`</code>	Extension persistence and masquerading anchor
Overlord-derived binaries	Linux AMD64, macOS Intel, macOS ARM hashes published by Proofpoint	Indicates RAT behavior, not only one-shot theft, on Linux/macOS
Windows encrypted payloads	<code>`.windows-js-pipeline.js.enc`</code> , <code>`.windows-agent-node.js.enc`</code> , <code>`.detect_malware.py.enc`</code>	Runtime decryption and editor-hosted execution complicate static controls

Hard-coded AES-GCM key	Proofpoint reports `4f7a8c3d2e1b5f9071a6b2c8d4e3f50a92b1c7d6e8f4a30b5c2d9e1f7a6b8c4d` for Windows encrypted files	Useful for sample confirmation if artifacts are obtained
Yeeth VSIX AES key	Shared C2 key `3e1c9a72f5b84d06e2a97c5310fb8e4d` across three reported marketplace extensions	Pivot for adjacent malicious extension family

5.2 Strings, Imports, and Embedded Artefacts

High-value strings and static pivots include `runOptions`, `folderOpen`, `vendor/run-update.sh`, `run-update-hidden-launch.vbs`, `ELECTRON\_RUN\_AS\_NODE`, `google-update-support`, `companywallet`, `browserlogin`, `cleanup`, `23.137.105.75`, `5173`, wallet extension IDs, browser profile paths, Keychain/Keyring access logic, `COM Elevation Moniker`, `IElevator2`, `AppComms\_Clients`, `AppComms\_Commands`, `AppComms\_Responses`, `investment-gateway.azurewebsites.net`, and `equity-core.azurewebsites.net`.

For adjacent malicious VSIX marketplace extensions, Yeeth Security reports JavaScript string encryption, Python and .NET worker agents, Roslyn scripting dependencies, forged PE timestamp, and a Microsoft Graph/SharePoint queue model. Static review of any recovered VSIX should prioritize `package.json` activation events, `extension.js`, `dist/` payloads, Graph API endpoints, encrypted string arrays, broker domains, and embedded Python/.NET execution engines.

5.3 Static Configuration Clues

Configuration Item	Reported / Expected Value	Confidence
UNK_DeadDrop C2	`23.137.105[.]75:5173`	High
Linux/macOS C2 protocol	Persistent WebSocket connection via Overlord-derived agent	High
Windows exfil protocol	HTTP POST to C2 after ZIP staging	High
VSIX persistence	Extension activates and re-launches Linux/macOS infection components	High
Windows persistence	No recurring VSIX persistence described; one-shot pipeline cleans up and terminates	Medium-high
Yeeth SharePoint site GUID	`e6bf72be-e8e2-4785-8814-5f874341d11f`	Medium-high
Yeeth Graph/SharePoint lists	`AppComms_Clients`, `AppComms_Commands`, `AppComms_Responses`	Medium-high

6.1 Reverse-Engineering Method

This task did not include original binary or VSIX samples. The code-analysis method is a source-backed reconstruction plus a reproducibility checklist for a future analyst who obtains artifacts.

Step	Tooling	Goal
Preserve repository	`git clone --mirror` in an isolated networkless VM; hash all files before opening	Avoid triggering tasks while preserving evidence
Pre-open scan	`jq`, `grep`, static scripts, YARA	Identify `.vscode/tasks.json`, `runOn: folderOpen`, shell/VBS/CMD launchers,

		VSIX packages
Unpack VSIX	`7z`, `unzip`	Inspect `package.json`, `extension.js`, `dist/`, activation events, embedded payloads
Reverse binaries	Ghidra, radare2, strings, capa, FLOSS	Confirm Overlord modules, C2 config, wallet/browser collection, fake password prompt logic
Decode scripts	CyberChef/Python, Node.js sandbox	Extract Base64, encrypted blobs, and AES-GCM configuration without execution
Dynamic validation	Disposable developer VM with fake wallets/secrets and controlled INetSim/sinkhole	Confirm process tree, file changes, C2 attempts, cleanup, credential access, and persistence

6.2 Functions and Logic of Interest

Function / Routine	Evidence	Why It Matters
VS Code task auto-run	`.vscode/tasks.json` launches platform-specific loader on folder open	Initial execution can be triggered by normal repository review behavior
VSIX installer	`run-update.sh` / `run-update.cmd` installs malicious extension	Turns repository execution into a persistent IDE foothold on Linux/macOS
Overlord `browserlogin` module	Proofpoint reports Chrome/Firefox credential theft	Direct credential risk on developer endpoints
Overlord `companywallet` module	Proofpoint reports wallet collection and ZIP upload	Primary financial-theft objective
Overlord `cleanup` module	Removes workspace payloads and malicious directories	Reduces forensic residue in the cloned repository
macOS fake password prompt	`darwin-password-prompt` and Keychain ACL modification	Enables keychain extraction and privilege escalation/relaunch as root
Linux Zenity prompt and GNOME Keyring access	Source reports Zenity and libsecret fallback	Socially obtains user password and accesses browser secrets
Windows Electron Node bootstrap	`ELECTRON_RUN_AS_NODE=1` runs JS inside editor runtime	Hides execution under Code/Cursor process and avoids dropping a Go RAT
Windows DPAPI/App-Bound bypass	`detect_malware.py.enc` performs COM Elevation Service / IElevator2 logic	Targets modern Chromium credential protections
Yeeth Roslyn REPL worker	`monitor-agent.exe` embeds Roslyn scripting	Allows arbitrary C# code execution without dropping a new DLL per command

6.3 Anti-Analysis, Evasion, and Obfuscation

The campaigns combine technical and social evasion. UNK_DeadDrop embeds payloads in realistic repositories, uses IDE auto-run features, installs a masqueraded VSIX, removes staging artifacts, runs Windows logic inside the editor's Electron runtime, and encrypts Windows payload blobs. Linux/macOS malware detaches the Overlord agent and preserves a persistent RAT connection through a VSIX re-launch mechanism. Yeeth Security's marketplace-extension reporting adds AES-encrypted strings, AES-encrypted Graph/SharePoint C2 payloads, low VirusTotal detection for a .NET worker, Costura.Fody

in-memory dependency loading, Roslyn dynamic execution, filename rotation, per-extension obfuscation keys, and Azure/Microsoft infrastructure abuse.

7.1 Execution Overview

Expected UNK_DeadDrop execution begins when a target opens the cloned repository in VS Code or Cursor. The hidden task starts the platform loader, which installs a VSIX extension and executes the platform payload. On macOS/Linux, the Overlord-derived Go binary establishes a persistent C2 channel, performs wallet collection, waits before credential theft, prompts for the user's password, attempts browser keychain/keyring access, relaunches with elevated privileges where possible, packages stolen material, and uploads it. On Windows, the hidden VBScript/CMD chain installs the VSIX, decrypts JS/Python payloads, launches the editor as Node via `ELECTRON\_RUN\_AS\_NODE=1`, collects wallets and browser credentials, uploads ZIP files by HTTP POST, cleans up, and terminates.

7.2 Filesystem, Registry, Service, and Task Changes

Artefact Type	Likely Location / Source	Triage Notes
Hidden workspace task	Cloned repository `.vscode/tasks.json`	Treat any unsolicited repository with `runOn: folderOpen` as suspicious until reviewed
Loader scripts	`vendor/run-update.sh`, `vendor/run-update-hidden-launch.vbs`, `run-update.cmd`	Hash and preserve; these may be removed by cleanup after execution
Malicious VSIX	VS Code/Cursor/VSCodium extension directories	Inventory extension IDs/names and install times across developer endpoints
Overlord agent binaries	Temporary/staged path selected by loader; extension-managed launch	Look for `google-update-support-*` filenames and hashes
Browser and wallet staging ZIPs	Runtime temp/profile paths before upload	May be short-lived; rely on EDR file events and command-line telemetry
Python staged under browser path	Proofpoint reports Windows may download Python into Chrome application directory	Unexpected Python under browser install directory is high-signal
Keychain/keyring access artifacts	macOS Keychain ACL changes; Linux secret-tool/Python D-Bus access	Correlate with fake password prompt and browser credential reads

7.3 Network and Command-and-Control Behaviour

Network Signal	Role	Confidence	Defensive Use
`23.137.105[.]75:5173`	UNK_DeadDrop C2 and exfil endpoint	High	Block, retrohunt, and correlate with Code/Cursor/VSCodium processes
WebSocket / WSS behavior	Persistent Linux/macOS Overlord channel	High	Hunt long-lived editor-adjacent connections to uncommon IPs/ports
HTTP POST ZIP upload	Windows wallet/credential exfiltration	High	Inspect proxy/EDR upload events from Code/Cursor/Electron processes
`investment-gateway[.]azurewebsites[.]net`	Yeeth-reported auth broker	Medium-high	Specific FQDN block and Graph audit correlation; avoid

			wildcard Azure blocking
`equity-core[.azurewebsites[.]net`	Yeeth-reported auth broker	Medium-high	Same as above
Microsoft Graph / SharePoint list traffic	Yeeth-reported C2 abstraction	Medium-high	Hunt for list names and site GUID in M365 audit/Graph telemetry

7.4 Observed Capability Summary

Capability	Supporting Dynamic Evidence
Remote access	Linux/macOS Overlord-derived agent maintains persistent WebSocket C2
Browser credential theft	Proofpoint reports Chrome/Firefox credential extraction and Chromium DPAPI/App-Bound bypass on Windows
Cryptocurrency wallet theft	Browser extension storage, standalone wallet apps, IndexedDB, wallet directories, and desktop wallets are targeted
Privilege escalation / credential prompting	macOS fake password prompt, Linux Zenity prompt, relaunch as root, Windows COM elevation/UAC fallback
Exfiltration	ZIP packaging and upload to `23.137.105[.]75:5173`; Yeeth extension supports large-file upload/download via encoded transfer
Anti-forensics	Cleanup of repository payloads and runtime files after execution
Marketplace persistence	VSIX activation/relaunch behavior in UNK_DeadDrop Linux/macOS and immediate activation in Yeeth-reported extensions

8. Process Tree and Execution Chain

Step	Parent Process	Child / Action	Why It Matters
1	Email client / browser	User opens attacker GitHub/GitLab repository link	Initial access is social engineering, not exploitation
2	`git` / browser / shell	Repository cloned to developer workstation	Malicious content lands as normal project files
3	`Code.exe`, `Cursor.exe`, `code`, `cursor`	Workspace opens and reads `.vscode/tasks.json`	IDE becomes the execution broker
4	VS Code/Cursor task runner	`/bin/bash vendor/run-update.sh` on Linux/macOS	Starts payload installation and Overlord staging
5	VS Code/Cursor task runner	`wscript.exe //B //Nologo vendor/run-update-hidden-launch.vbs` on Windows	Hidden script launches CMD and encrypted runtime payloads
6	Loader script	VSIX extension installed into VS Code/Cursor/VSCodium	Establishes IDE-native persistence/activation surface
7	Loader / editor	Overlord binary or	Platform-specific malware

		`ELECTRON_RUN_AS_NODE=1` pipeline starts	execution begins
8	Malware runtime	Browser/wallet/keychain/keyring/DPAPI collection	Credential and wallet theft phase
9	Malware runtime	POST/WebSocket to `23.137.105.75:5173`	Exfiltration and command channel
10	Cleanup routine	Removes `vendor/`, `.vscode/`, runtime files where configured	Reduces repository-local evidence

On Windows, analysts should not require a suspicious child process per command because the pipeline can run inside the editor's Electron runtime. On macOS/Linux, process telemetry may show detached `google-update-support-\*` binaries, password prompt components, Python helper processes, `secret-tool`, `runuser`, or browser profile/keyring access.

9. Persistence and System Modification

Modification Type	Artefact	Evidence	Confidence
VSIX extension persistence	`google-update-support.vsix` or similarly named extension	Proofpoint reports VSIX install and Linux/macOS relaunch behavior	High
IDE task execution	Workspace `.vscode/tasks.json`	Auto-run behavior on folder open	High
Linux/macOS RAT persistence	VSIX checks whether infection components are running and re-launches them	Proofpoint reporting	High
Windows one-shot operation	VSIX remains but does not re-execute on later editor starts	Proofpoint reporting	Medium-high
macOS Keychain ACL changes	Browser Safe Storage keys and login keychain access	Proofpoint reporting	High
Linux GNOME Keyring access	libsecret/secret-tool/Python D-Bus methods	Proofpoint reporting	High
Yeeth SharePoint beacon persistence	VSIX activation starts polling loop on IDE start	Yeeth reporting	Medium-high
Cleanup	Removal of payloads/directories from cloned repository	Proofpoint reporting	High

Remove the malicious extension and workspace artifacts, but do not treat removal as containment. Exfiltrated wallets, browser cookies, SSH keys, cloud/API tokens, package tokens, and source-control credentials require independent rotation or revocation.

10. Network and Infrastructure Analysis

IOC / Infrastructure	Type	Role	Stability / Confidence
`23.137.105[.]75:5173`	IP:port	UNK_DeadDrop C2 and exfil endpoint	High confidence; infrastructure may rotate
`170.205.29[.]83`,	IP addresses	Early sender/hosting	High confidence from

`170.205.30[.]227`		infrastructure	Proofpoint; historical value
Namecheap/MailHostBox/Mailgun sender domains	Email infrastructure	Recruitment/code-review phishing delivery	Medium-high; domain set likely ages quickly
Vercel-hosted fake sites	Web hosting	Fake company/project landing pages	Medium-high
GitHub/GitLab repositories	Code hosting	Payload delivery and lure credibility	High; repositories may be removed or recreated
Azure Websites broker domains	Cloud-hosted broker	Yeeth-reported Graph token broker for SharePoint C2	Medium-high
Microsoft Graph / SharePoint	Legitimate SaaS	Command queue and exfil abstraction in adjacent VSIX activity	Medium-high

The infrastructure pattern favors legitimate platforms and short-lived attacker domains. Blocking the listed indicators is necessary but not sufficient. Defenders should build behavior-centric controls around unsolicited developer projects, IDE auto-run tasks, extension installation, direct-IP editor egress, Graph/SharePoint command-queue misuse, and abnormal credential/wallet collection.

11. Indicators of Compromise

Machine-readable indicators are preserved in `iocs.csv`. The public IOC set is large; the table below highlights representative high-value entries and how to operationalize them.

IOC Type	Example / Count	Source	Operational Notes
C2 IP	`23.137.105[.].75`	Proofpoint	Block and retrohunt for editor or agent process connections to port 5173
Sender IP	2: `170.205.29[.]83`, `170.205.30[.]227`	Proofpoint	Historical mail/hosting pivots; correlate with inbound phishing
Domains	40+ sender/related domains	Proofpoint	Use for email gateway retrohunt and user notification scoping
Repository URLs	13 GitHub/GitLab URLs	Proofpoint	Identify cloned local paths, browser history, Git history, and endpoint file telemetry
SHA256	14 representative hashes	Proofpoint	Hash-blocking and sample confirmation for recovered artifacts
VSIX names	`ByteBinTools.jupyter-powerdev-2026.6.8.vsix`, `ToolCraft.jupyter-powertools-3.21.0.vsix`, `OLDev.markdown-mode-devtools-2.1.0.vsix`	Yeeth / THN	Extension inventory and marketplace governance
Graph/SharePoint C2 artifacts	Site GUID and list names	Yeeth	Microsoft 365 audit and Graph API threat hunting

Representative IOCs:

Type	Value	Description	Confidence
ip	23.137.105[.]75	UNK_DeadDrop C2 IP	High
ip	170.205.29[.]83	Sender/hosting infrastructure	High
ip	170.205.30[.]227	Sender/hosting infrastructure	High
domain	trixauvex[.]org	Sender/fake project infrastructure	High
domain	pulsynk[.]org	Sender/fake project infrastructure	High
domain	contacttrixauvex[.]link	Sender/fake project infrastructure	High
domain	mailpulsynk[.]xyz	Sender/fake project infrastructure	High
domain	predicttocareer[.]space	Sender/fake project infrastructure	High
domain	ondofinance[.]tech	Sender infrastructure spoofing legitimate brand	High
domain	nxlog[.]tech	Sender infrastructure spoofing legitimate brand	High
url	hxxps://github[.]com/Pulsynk/pulsynk	Attacker-controlled repository	High
url	hxxps://github[.]com/Trixauvex-org/trixauvex	Attacker-controlled repository	High
url	hxxps://github[.]com/PedrinPY/rekt-db	Attacker-controlled repository	High
url	hxxps://github[.]com/skyjum/x402-kit	Attacker-controlled repository	High
sha256	c935808147f0236c81483d7bbe da4b9d602f3595d5d4057f8115 d39e222d1c4b	`tasks.json`	High
sha256	734699773e53d995f20d485eb6 1261033d9d00b4332b39ca2607 1bcd60cd352f	`run-update.sh`	High
sha256	52886aab179f26421678ff23af1 b0fabf0a17ffbb534369cdbbac8 008cbcd8e7	`google-update-support.vsix`	High
sha256	a2b9a769df84d9d3a4694bb025 2a2c6a5e5f5d1a85a045653627 37092bbb3a86	`google-update-support-linux-amd64`	High

sha256	bb10adac5b0124efedfe71102c1d5638135ec9e1cde8c8cb3353c5ed91bb9f81	`google-update-support-darwin-amd64`	High
sha256	339907b44f161f57ff30819f422c552382ff437b3ae437463b4222cfe86bd943	`google-update-support-darwin-arm64`	High
sha256	808e7154b7af2bc7a4b28d577297c55f77221c355191cbe00f9f1810b6d4a619	`darwin-password-prompt`	High
vsix	ByteBinTools.jupyter-powerdev-2026.6.8.vsix	Malicious VS Code Marketplace extension	Medium-high
vsix	ToolCraft.jupyter-powertools-3.21.0.vsix	Malicious VS Code Marketplace extension	Medium-high
vsix	OLDev.markdown-mode-devtools-2.1.0.vsix	Malicious VS Code Marketplace extension	Medium-high
domain	investment-gateway[.]azurewebsites[.]net	Yeeth-reported Graph token broker	Medium-high
domain	equity-core[.]azurewebsites[.]net	Yeeth-reported Graph token broker	Medium-high
guid	e6bf72be-e8e2-4785-8814-5f874341d11f	Yeeth-reported SharePoint C2 site GUID	Medium-high

12. MITRE ATT&CK Mapping

Technique	Name	Evidence
T1566.002	Phishing: Spearphishing Link	Emails contain links to attacker-controlled GitHub/GitLab repositories
T1195.002	Supply Chain Compromise: Compromise Software Supply Chain	Malicious repositories and VSIX marketplace extensions abuse trusted developer workflows
T1204.002	User Execution: Malicious File	User opens/clones repository and opens it in VS Code/Cursor, triggering task execution
T1059.004	Command and Scripting Interpreter: Unix Shell	Linux/macOS loader uses `/bin/bash vendor/run-update.sh`
T1059.003	Command and Scripting Interpreter: Windows Command Shell	Windows chain runs CMD file after VBScript launch
T1059.005	Command and Scripting Interpreter: Visual Basic	Windows chain starts with `wscript.exe` executing a VBS launcher
T1059.007	Command and Scripting Interpreter: JavaScript	Windows pipeline runs JavaScript under Electron/Node; VSIX extension JavaScript handles C2 in adjacent activity

T1059.006	Command and Scripting Interpreter: Python	Windows credential stealer and Yeeth Linux/macOS agent use Python components
T1036	Masquerading	VSIX masquerades as Google service or Jupyter productivity tooling; infrastructure spoofs companies
T1555.003	Credentials from Web Browsers	Browser credentials, cookies, Safe Storage keys, and Chromium profiles targeted
T1555.001	Credentials from Password Stores: Keychain	macOS Keychain extraction and ACL modification reported
T1555.004	Credentials from Password Stores: Windows Credential Manager	Windows DPAPI/App-Bound browser credential access aligns with credential-store theft
T1056	Input Capture	Fake password prompts on macOS/Linux collect user credentials
T1005	Data from Local System	Wallet directories, browser profiles, desktop wallet apps, and files are collected
T1027	Obfuscated Files or Information	Base64 payloads, AES-encrypted Windows files, encrypted strings and C2 payloads
T1105	Ingress Tool Transfer	Repositories and VSIX packages deliver tooling; Yeeth extension can download additional payloads
T1071.001	Application Layer Protocol: Web Protocols	HTTP POST, WebSocket/WSS, Microsoft Graph API and SharePoint traffic
T1041	Exfiltration Over C2 Channel	ZIP uploads and SharePoint response/exfiltration channels move stolen data
T1070.004	Indicator Removal: File Deletion	Cleanup removes payloads and malicious repository artifacts

13.1 Detection Logic Summary

Signal	Telemetry Source	Analytic Goal	Tuning Notes
`vscode/tasks.json` with `runOn: folderOpen` in newly cloned repo	EDR file events, source-control scanning, repository pre-open hooks	Detect malicious or risky IDE auto-run tasks before execution	Some legitimate projects use auto tasks; tune on unsolicited repos, hidden paths, and network/script launchers
VS Code/Cursor spawning shell, wscript, cmd from repository path	EDR process events	Detect execution chain start	Baseline normal task usage for engineering teams
Editor process egress to `23.137.105.75:5173`	EDR network, proxy, firewall	Identify C2/exfiltration	High confidence; should be rare
VSIX install with suspicious names or `google-update-	Endpoint file inventory,	Find persistent IDE footholds	Include VS Code, Cursor, VSCodium and user-level

support`	extension inventory		extension paths
Browser credential/keychain access after IDE task execution	EDR, macOS ESF, Linux auditd, Windows credential events	Detect theft phase	Correlate by time and initiating process chain
Graph/SharePoint list names from VSIX runtime	M365 audit logs, proxy logs, Graph telemetry	Hunt adjacent malicious marketplace extensions	Legitimate Graph traffic is common; list names/site GUID improve precision

13.2 Detection Content

title: VS Code Or Cursor Auto Run Task Launches Script From Repository id: 83287e0c-8bf1-4e75-9b66-unk-deaddrop-folderopen-task status: experimental description: Detects VS Code or Cursor launching shell, VBScript, or CMD payloads from repository paths consistent with UNK_DeadDrop-style folderOpen task abuse. references: - https://www.proofpoint.com/us/blog/threat-insight/dont-fear-repo-unkdeaddrop-phishing-campaign-targets-developers-steal - https://thehackernews.com/2026/06/north-korean-hackers-are-turning.html author: Lost Boys Cyber date: 2026-06-17 logsource: category: process_creation product: windows detection: selection_parent: ParentImage endswith: - '\Code.exe' - '\Cursor.exe' - '\VSCodium.exe' selection_child: Image endswith: - '\wscript.exe' - '\cmd.exe' - '\powershell.exe' - '\pwsh.exe' - '\bash.exe' selection_args: CommandLine contains: - 'run-update' - '.vscode' - 'vendor\\' - 'ELECTRON_RUN_AS_NODE' condition: selection_parent and selection_child and selection_args falsepositives: - Legitimate IDE tasks in trusted internal repositories level: high
<pre>// Microsoft Defender XDR: editor processes connecting to UNK_DeadDrop C2 DeviceNetworkEvents where Timestamp > ago(180d) where RemoteIP == "23.137.105.75" or RemoteUrl has "23.137.105.75" where RemotePort == 5173 or RemoteUrl has ":5173" where InitiatingProcessFileName in~ ("Code.exe", "Cursor.exe", "VSCodium.exe", "code", "cursor") or InitiatingProcessCommandLine has_any ("google-update-support", "overlord", "ELECTRON_RUN_AS_NODE") project Timestamp, DeviceName, InitiatingProcessAccountName, InitiatingProcessFileName, InitiatingProcessCommandLine, RemoteIP, RemotePort, RemoteUrl, Protocol, ReportId</pre>
<pre>// Microsoft Defender XDR: suspicious VS Code/Cursor task auto-run files in developer workspaces DeviceFileEvents where Timestamp > ago(180d) where FolderPath has_any ("\\.vscode\\", "/.vscode/") where FileName =~ "tasks.json"</pre>


```

| where SHA256 in~ ("c935808147f0236c81483d7bbeda4b9d602f3595d5d4057f8115d39e222d1c4b")
| or PreviousFileName has "tasks.json"
| project Timestamp, DeviceName, InitiatingProcessAccountName, ActionType, FolderPath, FileName,
SHA256, InitiatingProcessFileName, InitiatingProcessCommandLine

// Microsoft Defender XDR: Code/Cursor spawning hidden script launchers from repository vendor paths
DeviceProcessEvents
| where Timestamp > ago(180d)
| where InitiatingProcessFileName in~ ("Code.exe", "Cursor.exe", "VSCodium.exe", "code", "cursor")
| where FileName in~ ("wscript.exe", "cmd.exe", "bash", "sh", "python.exe", "python3")
| where ProcessCommandLine has_any ("run-update", "run-update-hidden-launch.vbs", "run-update.cmd",
"vendor/run-update.sh", "ELECTRON_RUN_AS_NODE")
| project Timestamp, DeviceName, AccountName, InitiatingProcessFileName, InitiatingProcessCommandLine,
FileName, ProcessCommandLine, FolderPath, SHA256

index=edr (process_name="Code.exe" OR process_name="Cursor.exe" OR process_name="VSCodium.exe" OR
parent_process_name="Code.exe" OR parent_process_name="Cursor.exe")
((dest_ip="23.137.105.75" dest_port=5173) OR command_line="*run-update*" OR
command_line="*ELECTRON_RUN_AS_NODE*" OR command_line="*.vscode*folderOpen*")
| stats min(_time) as first_seen max(_time) as last_seen values(command_line) as command_lines
values(dest_ip) as dest_ips values(dest_port) as dest_ports by host user process_name
parent_process_name process_path
| convert ctime(first_seen) ctime(last_seen)

rule Suspicious_UNK_DeadDrop_Developer_Tool_Artifacts
{
    meta:
        description = "Detects repository, VSIX, and script artifacts associated with UNK_DeadDrop-
style developer tool malware delivery"
        author = "Lost Boys Cyber"
        date = "2026-06-17"
        confidence = "medium"
        caveat = "String rule for triage; validate with hashes, source context, and process/network
telemetry"
    strings:
        $task1 = "runOptions" ascii wide
        $task2 = "folderOpen" ascii wide
        $linux1 = "vendor/run-update.sh" ascii wide
        $win1 = "run-update-hidden-launch.vbs" ascii wide
        $win2 = "ELECTRON_RUN_AS_NODE" ascii wide
        $c2 = "23.137.105.75" ascii wide
        $vsix = "google-update-support" ascii wide
        $wallet = "companywallet" ascii wide
        $browser = "browserlogin" ascii wide
        $cleanup = "cleanup" ascii wide
    condition:
        ($task1 and $task2 and (any of ($linux*) or any of ($win*))) or
        ($c2 and any of ($vsix, $wallet, $browser, $cleanup)) or
        (3 of them)
}

```

13.3 Detection Validation and Blind Spots

Validate endpoint hits by confirming repository provenance, user intent, editor trust prompt behavior, VSIX install time, network destination, and whether browser/wallet files were accessed shortly after task execution. A detection hit on `runOn: folderOpen` alone is a triage lead, not proof of compromise; combine it with unsolicited repository source, encoded launchers, suspicious extension installation, direct-IP editor egress, or credential-store access.

Blind spots include macOS/Linux endpoints without full process lineage, developers using unmanaged Cursor installations, repositories deleted by cleanup, VSIX extensions removed before inventory, proxy logs without initiating process, encrypted Graph/SharePoint C2 that blends into allowed Microsoft 365 traffic, and variants that rotate C2 infrastructure or file names.

14.1 Hunt Hypotheses

- A developer opened an unsolicited repository in VS Code/Cursor and triggered `runOn: folderOpen` execution from a hidden `.vscode/tasks.json`.

- A developer endpoint installed a suspicious VSIX extension and then made direct-IP or Graph/SharePoint C2 connections.
- Browser, Keychain, GNOME Keyring, wallet, or DPAPI access occurred shortly after an IDE task execution event.
- Phishing recipients cloned or browsed one of the listed GitHub/GitLab repositories even if endpoint malware telemetry was later cleaned.
- A malicious marketplace extension used Microsoft Graph and SharePoint lists as a command queue, leaving M365 audit traces even when endpoint logs are sparse.

14.2 Hunt Queries

```
// Hunt 1: hidden VS Code task auto-run files created in developer workspaces
DeviceFileEvents
| where Timestamp > ago(180d)
| where FolderPath has_any ("./.vscode/", "\\.\vscode\\") and FileName =~ "tasks.json"
| join kind=leftouter (
    DeviceProcessEvents
    | where Timestamp > ago(180d)
    | where FileName in~ ("git.exe", "git", "Code.exe", "Cursor.exe", "VSCodium.exe", "code", "cursor")
    | project DeviceName, InitiatingProcessAccountName=AccountName, ProcTime=Timestamp, FileName,
ProcessCommandLine
) on DeviceName, InitiatingProcessAccountName
| summarize FirstSeen=min(Timestamp), Paths=make_set(FolderPath, 50),
RelatedCommands=make_set(ProcessCommandLine, 20) by DeviceName, InitiatingProcessAccountName

// Hunt 2: editor-origin script execution and credential-access follow-on
let EditorScriptEvents = DeviceProcessEvents
| where Timestamp > ago(180d)
| where InitiatingProcessFileName in~ ("Code.exe", "Cursor.exe", "VSCodium.exe", "code", "cursor")
| where FileName in~ ("wscript.exe", "cmd.exe", "bash", "sh", "python.exe", "python3", "osascript",
"security")
| where ProcessCommandLine has_any ("run-update", "vendor", "ELECTRON_RUN_AS_NODE", "secret-tool",
"keychain", "Login Data", "Local State");
EditorScriptEvents
| project Timestamp, DeviceName, AccountName, InitiatingProcessFileName, FileName, ProcessCommandLine,
InitiatingProcessCommandLine

// Hunt 3: Microsoft 365 / proxy logs for Yeeth-reported SharePoint list queue artifacts (adapt table
names to environment)
CloudAppEvents
| where Timestamp > ago(180d)
| where Application has_any ("Office 365", "SharePoint", "Microsoft Graph")
| where RawEventData has_any ("AppComms_Clients", "AppComms_Commands", "AppComms_Responses", "e6bf72be-
e8e2-4785-8814-5f874341d11f")
| project Timestamp, AccountDisplayName, AccountObjectId, Application, ActionType, IPAddress,
UserAgent, RawEventData

// Hunt 4: developer endpoint browser/wallet access shortly after Code/Cursor child-process execution
let SuspiciousEditorExec = DeviceProcessEvents
| where Timestamp > ago(180d)
| where InitiatingProcessFileName in~ ("Code.exe", "Cursor.exe", "VSCodium.exe", "code", "cursor")
| where ProcessCommandLine has_any ("run-update", "ELECTRON_RUN_AS_NODE", "google-update-support",
"secret-tool", "darwin-password-prompt");
DeviceFileEvents
| where Timestamp > ago(180d)
| where FolderPath has_any ("Login Data", "Local State", "Cookies", "Local Extension Settings",
"IndexedDB", "Exodus", "Electrum", "Ledger Live", "Monero", "Solana")
| join kind=innerunique (SuspiciousEditorExec | project DeviceName, AccountName, ExecTime=Timestamp,
ProcessCommandLine) on DeviceName, AccountName
| where Timestamp between (ExecTime .. ExecTime + 2h)
| project Timestamp, DeviceName, AccountName, FolderPath, FileName, ActionType, ExecTime,
ProcessCommandLine
```

14.3 Triage Guidance for Hunt Hits

A meaningful hit usually has at least two of the following: an unsolicited repository source, ``.vscode/tasks.json`` with ``runOn: folderOpen``, script launchers named ``run-update``, VSIX installation, Code/Cursor direct-IP egress to ``23.137.105.75:5173``, browser/wallet/keychain access shortly after editor execution, or Graph/SharePoint list artifacts matching the Yeeth-reported

C2 pattern. Escalate immediately if wallet directories, browser credential stores, cloud config files, package manager tokens, SSH keys, or source-control credentials were accessed.

Benign lookalikes include legitimate VS Code tasks in internal repositories, developer scripts that run on workspace open with explicit team approval, normal Microsoft Graph access by enterprise apps, and legitimate Jupyter/VSIX extension activity. Tuning should be based on repository origin, extension publisher, command-line content, network destination, and credential-access timing.

15. Triage and Incident Response

Priority	Action	Owner	Rationale
Critical	Isolate endpoints with confirmed execution, C2 contact, or credential-store access	SOC / IT	Prevent continued RAT access and exfiltration
Critical	Revoke and rotate exposed credentials	IAM / Cloud / DevOps	Browser, wallet, SSH, cloud, package registry, API, and source-control secrets may be stolen
High	Preserve cloned repository, VSIX files, EDR triage package, browser history, shell history, and extension inventory	IR / Forensics	Cleanup routines may remove evidence; preserve before remediation
High	Scope phishing recipients and repository access	Messaging / SOC	Identify additional users who may have cloned/opened projects
High	Review source-control and CI/CD audit logs	DevOps / AppSec	Developer endpoint compromise can lead to repository tampering or downstream package compromise
Medium	Block listed infrastructure and known hashes	Network / EDR	Reduces immediate known exposure, but do not rely on IOCs alone
Medium	Audit VS Code/Cursor trust and extension policy	Engineering / IT	Prevent recurrence through developer tooling governance

Collection priorities: EDR process/network/file telemetry, installed VS Code/Cursor/VSCodium extension directories, ``.vscode/tasks.json``, ``.vendor/`` scripts, Windows prefetch and PowerShell logs, macOS unified logs and Keychain events, Linux auditd/syslog and shell history, browser credential-store access telemetry, wallet directory file events, M365 Graph/SharePoint audit events, email headers, and Git clone/browser history.

Containment should include disabling suspicious VSIX extensions, removing malicious repository workspaces, terminating Overlord/editor child processes, blocking C2 IP/domain indicators, revoking tokens and credentials, invalidating active sessions, and monitoring for suspicious repository commits or package publications made after the exposure window.

16. Validation and Emulation

Validation completed for this report is source-backed: Lost Boys Cyber reviewed the intake article, extracted the Proofpoint primary report, reviewed Yeeth Security's primary malicious VSIX analysis, normalized IOCs, and produced detections/hunts mapped to reported behavior. No sample was executed locally because no malware archive was provided in the task.

Safe emulation steps for defenders:

Emulation Step	Method	Expected Telemetry
Repository pre-open scanner	Create benign test repo with <code>`.vscode/tasks.json`</code> and <code>`runOn: folderOpen`</code>	File detection and source-control policy alert without payload execution
IDE process analytic	Launch VS Code task that runs benign <code>`echo`/script</code> from a lab repo	Parent-child process detection from Code/Cursor to shell/script host
Network sinkhole validation	In lab only, simulate Code/Cursor process attempting connection to controlled sinkhole on port 5173	EDR/proxy alert for editor-origin suspicious egress
VSIX inventory	Install an internally built benign VSIX with known publisher/name	Extension inventory and approval workflow validation
Graph audit hunt	Generate benign SharePoint list events with controlled list names in a test tenant	M365 query validation without touching malicious infrastructure

Do not clone or open reported attacker repositories on a production workstation. If acquisition is required, use a network-isolated malware lab, disable IDE auto-run behavior, avoid opening the folder in VS Code/Cursor, and hash artifacts before any inspection.

17. Recommendations

Priority	Owner	Action	Rationale
Critical	Engineering / IT	Require isolated review for unsolicited recruitment/code-review repositories before opening in VS Code, Cursor, or similar IDEs	The initial execution boundary is normal developer behavior
Critical	SOC / Network	Block and retrohunt <code>`23.137.105.75`</code> , listed sender domains, and known hashes	Provides immediate coverage for reported UNK_DeadDrop indicators
Critical	IAM / DevOps	Rotate credentials on confirmed hosts: browser sessions, wallet seeds/keys, SSH keys, GitHub/GitLab tokens, package registry tokens, cloud keys, CI/CD secrets	Malware explicitly targets high-value developer secrets
High	AppSec / DevEx	Implement repository pre-open scanning for <code>`.vscode/tasks.json`</code> , <code>`runOn: folderOpen`</code> , suspicious launchers, and embedded VSIX packages	Prevents trust-on-open execution from unsolicited projects
High	IT / Engineering	Centrally govern VS Code, Cursor, and VSCodium extension installation; allowlist publishers and audit user-level extension directories	Malicious VSIX packages are both delivery and persistence mechanisms

High	SOC	Deploy process/network detections for Code/Cursor spawning script hosts and connecting to uncommon direct IPs or Graph/SharePoint command queues	Behavior-centric coverage survives IOC rotation
Medium	Messaging Security	Add detections for developer recruitment/code-review lures using GitHub/GitLab links, crypto project themes, and spoofed sender domains	Reduces initial access volume
Medium	M365 / Cloud	Monitor Graph API and SharePoint list activity for unusual developer-workstation user agents and known list names/GUIDs	Covers adjacent marketplace-extension C2 model
Medium	Security Awareness	Brief developers that opening a repository in an IDE can execute code; recommend plain-text/static review first for unknown projects	Closes the knowledge gap attackers exploit
Medium	IR / Legal	Define a crypto-wallet and developer-secret compromise playbook	Wallet theft and software-supply-chain blast radius require faster-than-normal response

18. References and Refresh Notes

- [1] Proofpoint, "Don't Fear the Repo: UNK_DeadDrop Phishing Campaign Targets Developers to Steal Cryptocurrency," published 2026-06-08. <https://www.proofpoint.com/us/blog/threat-insight/dont-fear-repo-unkdeaddrop-phishing-campaign-targets-developers-steal>
- [2] Yeeth Security, "Inside the DPRK-Linked Backdoor Loitering in the VS Code Marketplace," published 2026-06-09. <https://yeethsecurity.com/blog/2026-06-09-jupyter-powerdev-backdoor>
- [3] The Hacker News, "North Korean Hackers Are Turning Developer Tools Into Malware Delivery Channels," published 2026-06-15. <https://thehackernews.com/2026/06/north-korean-hackers-are-turning.html>
- [4] Microsoft, public reporting referenced by The Hacker News on Sapphire Sleet/BlueNoroff macOS social-engineering tradecraft.
- [5] Trend Micro, public reporting referenced by The Hacker News on Contagious Interview/Void Dokkaebi repository propagation and InvisibleFerret evolution.

Refresh notes: UNK_DeadDrop infrastructure, repository URLs, marketplace extension listings, and sender domains are likely to age quickly. Re-validate C2 reachability, repository status, marketplace extension status, and new hashes before operational blocking beyond the immediate response window. Attribution should also be refreshed as vendors publish additional telemetry linking or separating UNK_DeadDrop, Contagious Interview, Lazarus/Dream Job, BlueNoroff, and related DPRK developer-targeting clusters.